Práctica 2
Análisis sintáctico descendente

Santiago Zapata Amezcua

Compiladores 2027-1

1. Conjuntos N, Σ y símbolo inicial

N = { programa, declaraciones, declaracion, tipo, lista_var, sentencias, sentencia, expresion }

Σ = { int, float, identificador, numero, if, else, while, ‘;’, ‘,’, ‘=’, ‘(’, ‘)’, ‘+’, ‘-’, ‘*’, ‘/’ }

S = programa

En la regla de declaracion dice lista-var, pero es el mismo lista_var.

2. Eliminación de ambigüedad

Sí es ambigua y el problema está en expresion. Todas las operaciones están al mismo nivel, así que la gramática no dice cuál va primero. Por ejemplo, id + id * id se puede derivar de dos formas distintas. Aquí E es expresion e id es identificador.

$$E \Rightarrow E + E \Rightarrow id + E \Rightarrow id + E * E \Rightarrow id + id * E \Rightarrow id + id * id$$
$$E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow id + E * E \Rightarrow id + id * E \Rightarrow id + id * id$$

La primera agrupa como id + (id * id) y la segunda como (id + id) * id. Con id - id - id pasa lo mismo porque no se sabe si agrupar por la izquierda o por la derecha.

Para arreglarlo separé la expresión en niveles de precedencia. Abajo van los paréntesis, los identificadores y los números, luego * y /, y hasta arriba + y -. La recursión la dejé a la izquierda para que todo asocie por la izquierda, como en aritmética normal.

expresion	→	expresion + termino | expresion - termino | termino
termino	→	termino * factor | termino / factor | factor
factor	→	(expresion) | identificador | numero

Así id + id * id ya solo sale de una forma, porque el * queda dentro de un termino y se hace antes que la suma.

También hay ambigüedad con while e if-else. Su cuerpo es sentencias y no hay llaves, entonces si después de un while vienen dos asignaciones no se sabe si la segunda va dentro del while o después. Con lo que va después del else pasa igual. Lo resolví como el else colgante, el cuerpo se queda con todas las sentencias que siguen hasta que aparece un else o se acaba el programa. El parser ya lo hace así porque sigue leyendo sentencias mientras vea un identificador, un if o un while.

3. Eliminación de recursividad izquierda

Las reglas de declaraciones, lista_var, sentencias, expresion y termino tienen recursión izquierda directa porque empiezan llamándose a sí mismas, y eso hace que un parser descendente se cicle. Para quitarla usé la regla que vimos en clase.

$$
\begin{aligned}
A &\rightarrow A\alpha_1 \mid \ldots \mid A\alpha_m \mid \beta_1 \mid \ldots \mid \beta_n \\
&\text{se cambia por} \\
A &\rightarrow \beta_1 A' \mid \ldots \mid \beta_n A' \\
A' &\rightarrow \alpha_1 A' \mid \ldots \mid \alpha_m A' \mid \varepsilon
\end{aligned}
$$

En declaraciones α = declaracion y β = declaracion.

$$
\begin{aligned}
\text{declaraciones} &\rightarrow \text{declaraciones declaracion} \mid \text{declaracion} \\
&\Downarrow \\
\text{declaraciones} &\rightarrow \text{declaracion declaraciones}' \\
\text{declaraciones}' &\rightarrow \text{declaracion declaraciones}' \mid \varepsilon
\end{aligned}
$$

En lista_var α = , identificador y β = identificador.

$$
\begin{aligned}
\text{lista_var} &\rightarrow \text{lista_var , identificador} \mid \text{identificador} \\
&\Downarrow \\
\text{lista_var} &\rightarrow \text{identificador lista_var}' \\
\text{lista_var}' &\rightarrow \text{, identificador lista_var}' \mid \varepsilon
\end{aligned}
$$

En sentencias α = sentencia y β = sentencia.

$$
\begin{aligned}
\text{sentencias} &\rightarrow \text{sentencias sentencia} \mid \text{sentencia} \\
&\Downarrow \\
\text{sentencias} &\rightarrow \text{sentencia sentencias}' \\
\text{sentencias}' &\rightarrow \text{sentencia sentencias}' \mid \varepsilon
\end{aligned}
$$

En expresion α_1 = + termino, α_2 = - termino y β = termino.

$$
\begin{aligned}
\text{expresion} &\rightarrow \text{expresion + termino} \mid \text{expresion - termino} \mid \text{termino} \\
&\Downarrow \\
\text{expresion} &\rightarrow \text{termino expresion}' \\
\text{expresion}' &\rightarrow \text{+ termino expresion}' \mid \text{- termino expresion}' \mid \varepsilon
\end{aligned}
$$

En termino α_1 = * factor, α_2 = / factor y β = factor.

$$
\begin{aligned}
\text{termino} &\rightarrow \text{termino * factor} \mid \text{termino / factor} \mid \text{factor} \\
&\Downarrow \\
\text{termino} &\rightarrow \text{factor termino}' \\
\text{termino}' &\rightarrow \text{* factor termino}' \mid \text{/ factor termino}' \mid \varepsilon
\end{aligned}
$$

No hay recursión indirecta porque si sigues las reglas por la izquierda siempre llegas a un terminal sin volver al mismo no terminal.

4. Factorización izquierda

No hace falta porque ninguna regla tiene dos opciones que empiecen igual. La regla tipo empieza con int o float, sentencia con identificador, if o while, y factor con paréntesis, identificador o numero. En las reglas con prima cada opción empieza distinto o es ε. En la gramática original las opciones de expresion sí empezaban todas con expresion, pero eso ya se arregló en los pasos 2 y 3. Entonces con ver un solo token ya se sabe qué regla usar.

5. Nuevos conjuntos N y P

Σ y S no cambian.

N = { programa, declaraciones, declaraciones′, declaracion, tipo, lista_var, lista_var′, sentencias, sentencias′, sentencia, expresion, expresion′, termino, termino′, factor }

P = {

programa	→	declaraciones sentencias
declaraciones	→	declaracion declaraciones′
declaraciones′	→	declaracion declaraciones′ | ε
declaracion	→	tipo lista_var ;
tipo	→	int | float
lista_var	→	identificador lista_var′
lista_var′	→	, identificador lista_var′ | ε
sentencias	→	sentencia sentencias′
sentencias′	→	sentencia sentencias′ | ε
sentencia	→	identificador = expresion ;
	|	if (expresion) sentencias else sentencias
	|	while (expresion) sentencias
expresion	→	termino expresion′
expresion′	→	+ termino expresion′ | - termino expresion′ | ε
termino	→	factor termino′
termino′	→	* factor termino′ | / factor termino′ | ε
factor	→	(expresion) | identificador | numero

}